Bài tập lớn A1 – CNN và Transformer trên CIFAR-100

Học sâu và ứng dụng trong thị giác máy tính (CO5085) – Học kỳ 261 (2026–2027)

Sinh viên: Phan Ngọc Trâm – MSSV 2570790 (làm cá nhân)

Mã nguồn: https://github.com/tramphanngoc174/co5085-exercises (thư mục A1)

1. Tóm tắt

Báo cáo so sánh một CNN (ResNet-18) và một Transformer (ViT-Tiny/16) trên CIFAR-100 (100 lớp), với bốn chế độ huấn luyện: **scratch** (khởi tạo ngẫu nhiên), và ba chế độ trên trọng số pretrained: **head** (đóng băng toàn bộ backbone, chỉ train lớp phân loại), **partial** (đóng băng nửa đầu mạng) và **full** (fine-tune toàn bộ). Tổng cộng 8 cấu hình, mỗi cấu hình chạy một lần (seed 42, 15 epoch).

- Tốt nhất: **ViT-Tiny pretrained + fine-tune toàn bộ** với test accuracy 0,8754; ResNet-18 tốt nhất là full với 0,7982.
- **Từ đầu, CNN hơn hẳn Transformer**: ResNet-18 0,6038 so với ViT-Tiny 0,3911 (chênh 21,3 điểm).
- **Pretrain tăng mạnh**: full so với scratch tăng 19,4 điểm với ResNet-18 và 48,4 điểm với ViT-Tiny.
- **Đóng băng đổi accuracy lấy chi phí**: head-only chạy nhanh hơn khoảng 3 lần và dùng ít bộ nhớ hơn nhưng kém full nhiều; partial lấy lại phần lớn accuracy với chi phí thấp hơn full (mục 6).
- **Lưu ý quan trọng khi so sánh CNN và Transformer**: hai mô hình pretrained được pretrain trên dữ liệu khác nhau (ResNet-18 trên ImageNet-1k, ViT-Tiny trên ImageNet-21k rồi ImageNet-1k) và khác số tham số; do đó kết quả không tách riêng được ảnh hưởng của kiến trúc (mục 6.3, mục 8).

2. Dataset

Tập dữ liệu: CIFAR-100 (Krizhevsky, 2009): 60.000 ảnh màu 32×32, 100 lớp (600 ảnh mỗi lớp), gồm 50.000 ảnh train và 10.000 ảnh test gốc. Lý do chọn: nằm trong danh sách đề cho phép; nhiều lớp (100) và khó hơn hẳn các tập của E1; đủ nhỏ để chạy 8 cấu hình trên GPU Colab; có nhiều cặp lớp giống nhau (các loại cây, các nhóm người, đồ dùng) nên phân tích lỗi có ý nghĩa.

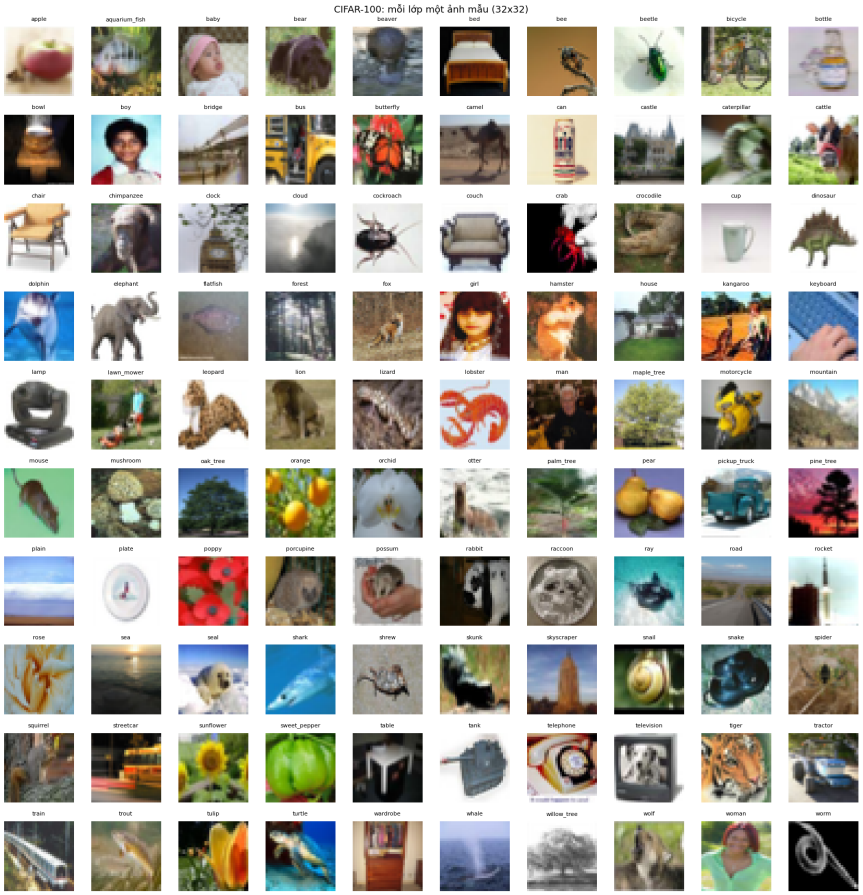
License: đây là tập dữ liệu công khai do Krizhevsky công bố, trích dẫn bài báo “Learning Multiple Layers of Features from Tiny Images” (2009). Thẻ dataset chính thức trên Hugging Face ghi license là “unknown”; một số bản sao của bên thứ ba ghi MIT hoặc CC BY 4.0 nhưng đó không phải nguồn gốc. Nhóm chỉ dùng cho học tập, phi thương mại; người đọc nên đối chiếu thêm tại trang chính thức (cs.toronto.edu/~kriz/cifar.html).

Chia dữ liệu (seed 42, chỉ số lưu trong splits/cifar100_tr450_val50_seed42.json):

Tập	Số ảnh	Mỗi lớp	Nguồn
Train	45.000	450	tập train gốc (50.000 ảnh)
Validation	5.000	50	tập train gốc, không trùng train; dùng để chọn epoch
Test	10.000	100	tập test gốc, chỉ dùng để báo cáo cuối

Cả 50.000 ảnh train gốc đều được dùng (không lấy subset). Các lớp cân bằng hoàn toàn nên accuracy và macro-F1 gần nhau.

EDA: ảnh gốc 3×32×32; trung bình RGB trên tập train (0,5068; 0,4865; 0,4412), độ lệch chuẩn (0,2674; 0,2565; 0,2761). Hình 1 là một ảnh mẫu mỗi lớp. Ảnh rất nhỏ nên nhiều lớp (ví dụ các loại cây, boy/girl/man/woman) khó phân biệt kể cả bằng mắt.



Hình 1. CIFAR-100: mỗi lớp một ảnh mẫu (32×32).

3. DataLoader và augmentation

- Dataset: lớp TensorImageDataset giữ toàn bộ ảnh dưới dạng tensor uint8 (N×3×32×32) trong bộ nhớ; DataLoader batch size 128, tập train xáo trộn bằng generator có seed (file A1/data.py).
- Augmentation chỉ ở tập train, chạy trên GPU cho từng batch: **random crop** 32×32 sau khi đệm 4 pixel giá trị 0 mỗi bên (độ lệch ngẫu nhiên riêng cho từng ảnh) và **lật ngang** với xác suất 0,5. Tập validation và test không augmentation.
- Sau augmentation, ảnh được **phóng to bằng nội suy bilinear từ 32×32 lên 224×224** (đúng kích thước pretrain của hai mô hình), rồi chuẩn hóa mean/std theo cấu hình pretrained của timm cho từng mô hình. Phóng to không thêm thông tin mới cho ảnh; đây là một lựa chọn để dùng trọng số pretrained.
- Huấn luyện dùng mixed precision (fp16 autocast + GradScaler) để tiết kiệm thời gian và bộ nhớ.

4. Protocol thí nghiệm

Mô hình (thư viện timm 1.0.31, torch 2.14.1+cu130 theo config.json):

Kiến trúc	Họ	Tham số (100 lớp)	Trọng số pretrained (timm)	Pretrain trên
ResNet-18	CNN	11.227.812	resnet18.a1_in1k	ImageNet-1k
ViT-Tiny/16	Transformer	5.543.716	vit_tiny_patch16_224.augreg_in21k_ft_in1k	ImageNet-21k, rồi fine-tune ImageNet-1k

Bốn chế độ huấn luyện (trên từng kiến trúc):

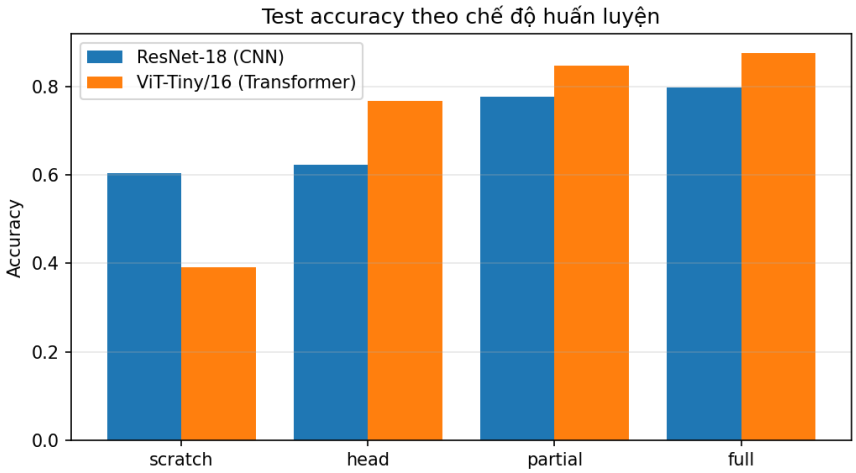
Chế độ	Trọng số khởi tạo	Phần được train	ResNet-18 trainable	ViT-Tiny trainable
scratch	ngẫu nhiên	toàn bộ	11.227.812 (100%)	5.543.716 (100%)
head	pretrained	chỉ lớp phân loại (backbone đóng băng)	51.300 (0,5%)	19.300 (0,3%)
partial	pretrained	ResNet: layer3, layer4, fc (đóng băng conv1, bn1, layer1, layer2). ViT: block 6–11, norm, head (đóng băng patch_embed, cls_token, pos_embed, block 0–5)	10.544.740 (93,9%)	2.688.868 (48,5%)
full	pretrained	toàn bộ	11.227.812 (100%)	5.543.716 (100%)

- Tối ưu: AdamW (weight decay 0,01), lịch cosine theo epoch trong 15 epoch, loss Cross-Entropy. Learning rate: scratch 1e-3; head 1e-3 cho lớp phân loại; partial và full dùng 1e-4 cho backbone và 1e-3 cho lớp phân loại. Chưa tinh chỉnh các siêu tham số này.
- Các lớp BatchNorm của phần bị đóng băng được giữ ở chế độ eval để thống kê chạy không bị cập nhật; mã kiểm thử A1/smoke_test.py có bước kiểm tra trọng số và BatchNorm đóng băng không đổi sau một epoch.
- Vòng lặp huấn luyện tự viết (zero_grad → forward → loss → backward → step). Chọn epoch có validation accuracy cao nhất, rồi đo test accuracy một lần. Chỉ số: accuracy, macro-F1, số tham số trainable, thời gian mỗi epoch (gồm cả lượt validation), bộ nhớ GPU đỉnh. Mỗi cấu hình chạy một lần, seed 42.

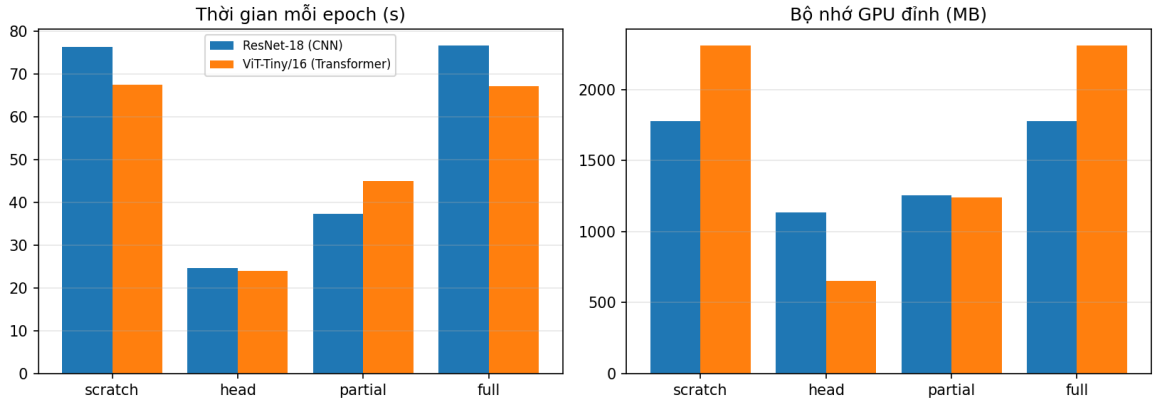
5. Kết quả

Kiến trúc	Chế độ	Trainable	Best ep.	Val acc	Test acc	Macro F1	s/epoch	Tổng (s)	Bộ nhớ (MB)
ResNet-18	scratch	11.227.812	15	0,6098	0,6038	0,6030	76,3	1144	1779
ResNet-18	head	51.300	14	0,6290	0,6233	0,6206	24,6	369	1135
ResNet-18	partial	10.544.740	13	0,7736	0,7780	0,7773	37,3	559	1256
ResNet-18	full	11.227.812	13	0,7924	0,7982	0,7981	76,6	1150	1778
ViT-Tiny	scratch	5.543.716	15	0,3802	0,3911	0,3829	67,6	1013	2309
ViT-Tiny	head	19.300	12	0,7668	0,7675	0,7673	24,0	360	651
ViT-Tiny	partial	2.688.868	12	0,8438	0,8478	0,8475	44,9	674	1241
ViT-Tiny	full	5.543.716	14	0,8730	0,8754	0,8753	67,2	1008	2309

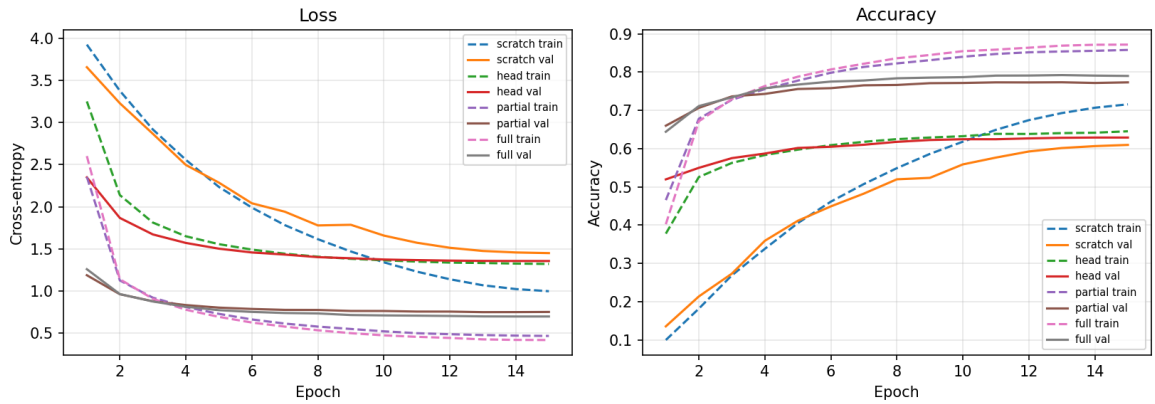
Nguồn: results/A1/summary.md (một lần chạy cho mỗi cấu hình; test accuracy đo tại epoch có validation accuracy cao nhất).



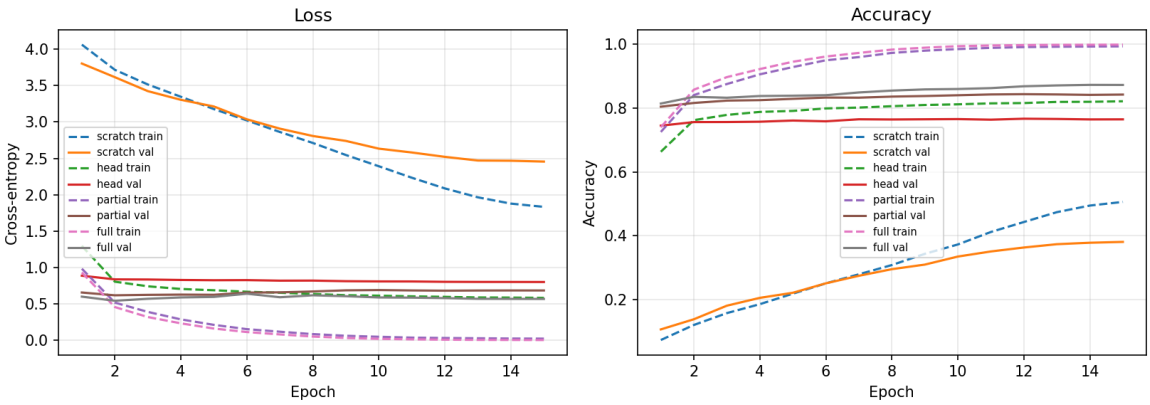
Hình 2. Test accuracy theo chế độ huấn luyện và kiến trúc.



Hình 3. Thời gian mỗi epoch và bộ nhớ GPU đỉnh.



Hình 4. Đường cong loss/accuracy của ResNet-18 (nét đứt: train, nét liền: validation).



Hình 5. Đường cong loss/accuracy của ViT-Tiny.

6. So sánh và thảo luận

6.1 Train từ đầu và pretrained

Kiến trúc	scratch (test acc)	full (test acc)	Chênh lệch
ResNet-18	0,6038	0,7982	+19,44 điểm
ViT-Tiny	0,3911	0,8754	+48,43 điểm

- Pretrain giúp cả hai mô hình, nhưng ViT-Tiny được lợi nhiều hơn rất nhiều (khoảng 48 điểm so với 19 điểm).
- **ViT cần pretrain trong thiết lập này.** Từ đầu, ViT-Tiny chỉ đạt 0,3911 trên 45.000 ảnh, thấp hơn ResNet-18 từ đầu 21,3 điểm; sau pretrain thì ViT-Tiny lại vượt ResNet-18. Điều này phù hợp với nhận định thường gặp rằng Transformer cần nhiều dữ liệu hơn khi không có tri thức đã học, nhưng thí nghiệm này chưa kiểm chứng riêng (chẳng hạn bằng cách thay đổi lượng dữ liệu).
- Cả hai mô hình scratch đều chưa hội tụ sau 15 epoch: best epoch là 15 và validation accuracy vẫn tăng (Hình 4–5). Do đó khoảng cách scratch và pretrained có thể bị phóng đại so với khi huấn luyện lâu hơn.

6.2 Các kiểu đóng băng

Kiến trúc	Chế độ	Test acc	So với full	s/epoch (% của full)	Bộ nhớ đỉnh (% của full)
ResNet-18	head	0,6233	-17,49 điểm	24,6 (32%)	1135 (64%)
ResNet-18	partial	0,7780	-2,02 điểm	37,3 (49%)	1256 (71%)
ResNet-18	full	0,7982	–	76,6 (100%)	1778 (100%)
ViT-Tiny/16	head	0,7675	-10,79 điểm	24,0 (36%)	651 (28%)
ViT-Tiny/16	partial	0,8478	-2,76 điểm	44,9 (67%)	1241 (54%)
ViT-Tiny/16	full	0,8754	–	67,2 (100%)	2309 (100%)

- **Head-only** rẻ nhất: khoảng 24 giây/epoch (so với 67–77 giây của full) và ViT-Tiny chỉ cần 651 MB. Với ViT-Tiny, head-only vẫn đạt 0,7675 chỉ với 19.300 tham số trainable, cao hơn ResNet-18 scratch và gần ResNet-18 partial (0,7780). Với ResNet-18, head-only chỉ đạt 0,6233, hơn scratch có 1,9 điểm: đặc trưng ImageNet đóng băng của ResNet-18 trên ảnh CIFAR phóng to dùng được kém hơn đặc trưng của ViT-Tiny.
- **Partial** lấy lại gần hết accuracy của full (kém 2,0 điểm ở ResNet-18 và 2,8 điểm ở ViT-Tiny) với chi phí thấp hơn: ResNet-18 chỉ mất khoảng một nửa thời gian (dù vẫn train 93,9% số tham số, vì phần đóng băng là các tầng đầu), ViT-Tiny mất khoảng 2/3 thời gian và dùng

khoảng 54% bộ nhớ của full.

- **Full** cho accuracy cao nhất ở cả hai kiến trúc nhưng đắt nhất. Freeze có lợi khi ưu tiên chi phí hoặc khi dữ liệu/tài nguyên hạn chế (partial là điểm cân bằng tốt trong thí nghiệm này); khi cần accuracy tối đa và đủ tài nguyên thì fine-tune toàn bộ.
- Chênh lệch giữa partial và full chỉ 2–3 điểm, có thể gần với độ dao động giữa các lần chạy (mục 8), nên không nên xem là khác biệt chắc chắn.

6.3 CNN và Transformer

Chế độ	ResNet-18	ViT-Tiny	ViT − ResNet
scratch	0,6038	0,3911	-21,27 điểm
head	0,6233	0,7675	+14,42 điểm
partial	0,7780	0,8478	+6,98 điểm
full	0,7982	0,8754	+7,72 điểm

- Ở cả ba chế độ pretrained, ViT-Tiny cao hơn ResNet-18 (từ khoảng 7 đến 14 điểm); ở scratch thì ngược lại, ResNet-18 cao hơn nhiều.
- **Không thể quy chênh lệch pretrained cho riêng kiến trúc.** ViT-Tiny được pretrain trên ImageNet-21k rồi ImageNet-1k, còn ResNet-18 chỉ trên ImageNet-1k, và hai mô hình khác công thức pretrain (augreg và a1) cũng như số tham số (5,5 triệu so với 11,2 triệu). Kết quả scratch là phép so sánh sạch hơn (cả hai không có pretrain) và cho thấy CNN dễ huấn luyện hơn trong điều kiện dữ liệu và thời gian của bài này.
- Về chi phí: ở scratch và full, ViT-Tiny nhanh hơn một chút (khoảng 67 so với 76 giây/epoch) dù chỉ có khoảng một nửa số tham số, nhưng dùng nhiều bộ nhớ đỉnh hơn (2309 MB so với 1778 MB); ở head-only ViT-Tiny dùng ít bộ nhớ hơn (651 MB so với 1135 MB).

6.4 Overfitting

- Các mô hình pretrained của ViT-Tiny khớp tập train gần như hoàn toàn (train accuracy 0,9993 với full và 0,9938 với partial ở epoch 15) trong khi validation accuracy là 0,8728 và 0,8426, tức khoảng cách khoảng 12,7 và 15,1 điểm. Validation loss của partial thấp nhất ở epoch 2 (0,6197) rồi tăng lên 0,6860, trong khi validation accuracy vẫn nhích lên.
- ResNet-18 pretrained có khoảng cách nhỏ hơn (8,5 điểm với partial, 8,2 điểm với full). Các biện pháp chống overfitting (augmentation mạnh hơn, dropout, weight decay lớn hơn, dừng sớm) chưa được thử.

7. Phân tích lỗi

Dưới đây là các cặp lớp bị nhầm nhiều nhất của cấu hình tốt nhất mỗi kiến trúc (full), tính trên 100 ảnh test mỗi lớp (nguồn: results/A1/top_confusions.md).

ResNet-18 full: thật → đoán	Số ảnh	ViT-Tiny full: thật → đoán	Số ảnh
seal → otter	24	oak_tree → maple_tree	21
maple_tree → oak_tree	17	maple_tree → oak_tree	15
pine_tree → oak_tree	16	girl → baby	11
girl → woman	16	girl → boy	11
woman → girl	16	woman → man	11
dolphin → shark	15	boy → baby	10
bowl → plate	15	dolphin → whale	10
boy → man	15	plate → bowl	10

ResNet-18 full: thật → đoán	Số ảnh	ViT-Tiny full: thật → đoán	Số ảnh
woman → man	14	willow_tree → maple_tree	10
boy → baby	13	boy → man	9

- Các lỗi tập trung vào cặp lớp giống nhau về hình dạng hoặc ngữ nghĩa: các loại cây (maple, oak, pine, willow), nhóm người (boy, girl, man, woman, baby), đồ dùng (bowl và plate), động vật sống nước (dolphin với shark hoặc whale, seal với otter). Với ảnh 32×32, chi tiết phân biệt (hình dạng lá, tuổi, giới tính) gần như bị mất.
- Hai mô hình có chung nhiều cặp nhầm (maple_tree và oak_tree, girl và boy/woman, boy và man, bowl và plate, dolphin với cá voi hoặc cá mập), nên khó khăn đến từ dữ liệu hơn là từ riêng một kiến trúc. Nhầm lẫn lớn nhất của ResNet-18 là seal → otter (24 trên 100 ảnh seal).

8. Hạn chế và tính minh bạch

- **Mỗi cấu hình chỉ chạy một lần.** Lần chạy đầu của vit_tiny scratch bị gián đoạn (Colab ngắt) ở epoch 10 và được chạy lại từ đầu với cùng seed 42. Ở epoch 10, validation accuracy của lần bị gián đoạn là 0,3054 còn lần chạy lại là 0,3342 (chênh 2,9 điểm). Điều này cho thấy kết quả không hoàn toàn lặp lại giữa các lần chạy dù cùng seed (có thể do tính không tất định của GPU), nên các chênh lệch dưới khoảng 3 điểm cần được xem thận trọng. Bảng kết quả dùng lần chạy hoàn chỉnh.
- **Yếu tố gây nhiễu giữa CNN và Transformer**: nguồn pretrain khác nhau (ImageNet-1k so với ImageNet-21k rồi 1k), số tham số khác nhau, công thức pretrain khác nhau. Báo cáo không thực hiện ablation để tách các yếu tố này.
- Huấn luyện 15 epoch cho mọi cấu hình; các mô hình scratch chưa hội tụ nên accuracy của chúng là cận dưới. Chưa tinh chỉnh learning rate hay siêu tham số khác.
- Ảnh 32×32 được phóng lên 224×224 để dùng mô hình pretrained; kết quả có thể khác nếu dùng ảnh độ phân giải thật.
- Chỉ so sánh một CNN và một Transformer cỡ nhỏ; kết luận không nên khái quát sang các kiến trúc hoặc kích thước khác.
- Chưa thử các biện pháp chống overfitting và chưa đo thời gian suy luận.

9. Tái lập

Từ thư mục gốc của repo (trên Colab, cài thêm timm): pip install timm, python A1/eda.py, python A1/run_a1.py. Mặc định chạy 8 cấu hình, 15 epoch, ảnh 224×224. Mỗi cấu hình lưu vào results/A1/<kiến trúc>_<chế độ>/ (history.json, confusion.npy, result.json); chạy lại cùng lệnh sẽ bỏ qua các cấu hình đã xong. Cấu hình, log và bảng nằm ở results/A1/ (config.json, log.txt, summary.md, top_confusions.md).

10. Ghi nhận việc sử dụng công cụ AI

Công cụ AI Claude (Anthropic) được dùng để hỗ trợ phân tích đề, soạn khung mã nguồn A1 (thư mục A1) và bản nháp báo cáo, trang GitHub Pages từ số liệu thật do sinh viên cung cấp. Sinh viên chạy thí nghiệm trên Google Colab, cung cấp log và hình kết quả, và chịu trách nhiệm kiểm tra, giải thích mã nguồn và số liệu đã nộp.